GRUPO 7 · REQUERIMIENTOS FUNCIONALES

Carrito de Compras / E-commerce

Módulo: Checkout de un e-commerce

Curso de Automatización de Pruebas de Software
Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 7
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el cierre de compra: la creación de una orden a partir de un conjunto de ítems, el cálculo del importe total del lado del servidor y la consulta de la orden con su detalle. La orden y sus ítems se escriben dentro de una misma transacción.

Fuera de alcance: Catálogo de productos, control de existencias, carrito persistente antes del cierre, cupones y descuentos, costos de envío, cobro del importe y cambios de estado posteriores de la orden.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/ordenes	RF-G7-01	Lista órdenes activas, opcionalmente filtradas por comprador.
POST /api/v1/ordenes	RF-G7-02	Cierra la compra: crea la orden con sus ítems.
GET /api/v1/ordenes/{id}	RF-G7-03	Devuelve una orden con el detalle de sus ítems.
PUT /api/v1/ordenes/{id}	RF-G7-04	Recalcula producto/monto de la orden a partir de ítems (no toca `items_orden`).
DELETE /api/v1/ordenes/{id}	RF-G7-05	Da de baja (soft-delete) una orden.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.

- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- El cierre de compra requiere un comprador existente; el `usuarioId` se obtiene del alta de cliente (RF-G4-01) o de los datos sembrados.
- Los importes de los ítems deben enviarse como números en el cuerpo JSON, no como texto entre comillas: este endpoint no convierte textos numéricos.

3. Modelo de datos del módulo

ordenes

Cabecera de la compra: comprador, producto representativo, importe total y estado.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`producto`	`text`, obligatorio; la API guarda el producto del primer ítem
`monto`	`numeric(10,2)`, obligatorio; lo calcula el servidor
`estado`	`text`, uno de `pendiente` / `pagada` / `enviada` / `cancelada`; por defecto `pendiente`
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

items_orden

Detalle de la compra: una fila por producto comprado.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`orden_id`	`bigint`, obligatorio, referencia a `ordenes(id)`
`producto`	`text`, obligatorio
`cantidad`	`integer`, obligatorio, la base exige que sea mayor que cero
`precio_unitario`	`numeric(10,2)`, obligatorio
`subtotal`	`numeric(10,2)`, obligatorio; lo calcula el servidor
`created_at`	`timestamptz`, por defecto `now()`

> El importe total de la orden y el subtotal de cada ítem los calcula el servidor a partir de la cantidad y el precio unitario recibidos. Un total enviado por el cliente sería ignorado: no existe ese campo en la entrada.

4. Requerimientos funcionales

RF-G7-01 — Listar órdenes

`GET /api/v1/ordenes`

Devuelve las órdenes del sandbox, con la posibilidad de acotar el resultado a un comprador determinado. La lista no incluye el detalle de ítems.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente órdenes con `activo = true`; una dada de baja con RF-G7-05 deja de aparecer.
- Con `usuarioId`, devuelve todas las órdenes de ese comprador ordenadas por identificador ascendente.
- Sin `usuarioId`, devuelve las primeras 100 órdenes del sandbox.
- La respuesta contiene solo las cabeceras: para ver los ítems hay que consultar la orden puntual (RF-G7-03).
- Un comprador sin órdenes devuelve una lista vacía.

Respuesta esperada

- `200 OK` con `data` como arreglo de órdenes, cada una con `id`, `usuario_id`, `producto`, `monto`, `estado` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/ordenes/route.ts`

RF-G7-02 — Cerrar la compra (checkout)

`POST /api/v1/ordenes`

Crea una orden a partir de la lista de ítems comprados. El sistema calcula los subtotales y el importe total, y guarda la cabecera y el detalle en una sola operación indivisible.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del comprador.
- `items` (obligatorio): arreglo con al menos un elemento.
- `items[].producto` (obligatorio): texto no vacío.
- `items[].cantidad` (obligatorio): número entero mayor que cero, enviado como número.
- `items[].precioUnitario` (obligatorio): número mayor que cero, enviado como número.

Reglas de negocio

- El subtotal de cada ítem se calcula como cantidad por precio unitario; el cliente no lo envía.
- El importe total de la orden es la suma de los subtotales de todos los ítems: **nunca se confía en un total enviado por el cliente**.
- El campo `producto` de la cabecera se completa con el producto del primer ítem del arreglo, como resumen de la compra.
- La orden se crea siempre con `estado = 'pendiente'`.
- La cabecera y todos los ítems se escriben dentro de una misma transacción: si falla la inserción de un ítem, tampoco queda la orden.
- Una compra sin ítems es rechazada: el arreglo debe tener al menos un elemento.
- Cantidad y precio unitario deben ser números en el cuerpo JSON; enviados como texto entrecomillado, la compra se rechaza por validación.
- **No hay control de existencias, ni catálogo, ni precios de referencia:** el producto es texto libre y el precio lo fija quien compra.
- El cierre de compra no cobra el importe ni descuenta saldo de ninguna cuenta o tarjeta.

Respuesta esperada

- `201 Created` con `data` conteniendo la orden creada y, dentro de la propiedad `items`, cada ítem con su `subtotal` ya calculado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `usuarioId`, el arreglo de ítems está vacío o ausente, un ítem tiene cantidad o precio no positivos o enviados como texto, o el comprador indicado no existe.
`EXECUTION_ERROR`	400	El importe total supera la capacidad numérica del campo `monto`.

Fuente: `app/api/v1/ordenes/route.ts`

RF-G7-03 — Consultar una orden con su detalle

`GET /api/v1/ordenes/{id}`

Devuelve la cabecera de una orden junto con todos sus ítems, para verificar el contenido de la compra.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la orden cuyo identificador coincide exactamente, junto con sus ítems ordenados por identificador ascendente.
- Si la orden no existe, responde 404 con el mensaje "Orden no encontrada.".
- Una orden sin ítems (posible solo en datos cargados a mano) devuelve la cabecera con un arreglo de ítems vacío.

Respuesta esperada

- `200 OK` con `data` conteniendo los campos de la orden más la propiedad `items` con el detalle completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una orden con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/ordenes/[id]/route.ts`

RF-G7-04 — Recalcular una orden

`PUT /api/v1/ordenes/{id}`

Recalcula el producto representativo y el importe total de una orden a partir de una lista de ítems enviada de nuevo. **No modifica las filas de `items_orden`**: el rol `qa_api` no tiene permiso `DELETE`, así que no hay forma de reemplazar el detalle sin dejar filas huérfanas, y este endpoint no inserta ítems nuevos tampoco. Solo actualiza los dos campos propios de la cabecera.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `items` (obligatorio): arreglo con al menos un elemento, con la misma forma que en el checkout (`producto`, `cantidad`, `precioUnitario`).

Reglas de negocio

- El `monto` se recalcula como la suma de cantidad por precio unitario de los ítems enviados, igual que en el checkout.
- El `producto` de la cabecera se reemplaza por el del primer ítem del arreglo enviado.
- **Los ítems enviados nunca se guardan en `items_orden`:** tras este `PUT`, `GET /ordenes/{id}` sigue devolviendo el detalle de ítems original de la compra, que ya no coincide con el nuevo `monto` de la cabecera.
- Solo puede recalcular una orden con `activo = true`; una dada de baja responde 404.
- `estado` no forma parte del cuerpo: no cambia por este reemplazo.

Respuesta esperada

- `200 OK` con `data` conteniendo la cabecera de la orden ya actualizada (sin la propiedad `items`).

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una orden vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta `items`, el arreglo está vacío, o algún ítem tiene cantidad o precio no positivos.

Fuente: `app/api/v1/ordenes/[id]/route.ts`

RF-G7-05 — Dar de baja una orden

`DELETE /api/v1/ordenes/{id}`

Da de baja lógica una orden, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- No borra ni afecta las filas de `items_orden` asociadas: quedan huérfanas (ya no accesibles por `GET /ordenes/{id}`, que responde 404).
- Una orden en cualquier `estado` de negocio puede darse de baja.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una orden vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/ordenes/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- **`DELETE` es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.

- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones` , `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/ordenes` — Listado de órdenes	Tabla con producto, importe y estado, con filtro por comprador. Ejecuta RF-G7-01.
`/ordenes/new` — Nueva orden	Formulario con el comprador y una tabla editable de ítems (producto, cantidad, precio unitario), donde se pueden agregar filas. Ejecuta RF-G7-02 y, al confirmarse, navega al detalle de la orden.
`/ordenes/{id}` — Detalle de la orden	Muestra la cabecera y la tabla de ítems con su subtotal. Ejecuta RF-G7-03.

- El formulario convierte a número la cantidad y el precio antes de enviarlos, de modo que el error por enviarlos como texto solo puede provocarse llamando la API directamente.
- El total de la compra no se edita en la pantalla: se ve recién en el detalle, ya calculado por el servidor.
- No hay pantalla de edición ni baja: `PUT` y `DELETE /ordenes/{id}` (RF-G7-04/05) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G7-01 — Listar órdenes

- Al listar órdenes filtrando por un comprador, todos los elementos devueltos tienen ese `usuario_id`.
- Los elementos del listado no incluyen el arreglo de ítems.
- Al filtrar por un comprador sin órdenes, la respuesta es 200 con `data` vacío.
- Una orden recién creada aparece en el listado de su comprador.

RF-G7-02 — Cerrar la compra (checkout)

- Al cerrar una compra con dos ítems, la respuesta es 201 y `data.monto` es igual a la suma de cantidad por precio unitario de ambos ítems.
- Cada elemento de `data.items` trae su `subtotal` calculado por el servidor, coincidente con su cantidad por su precio unitario.
- El campo `data.producto` de la cabecera coincide con el producto del primer ítem enviado.
- La orden creada queda en estado `pendiente`.

- Al cerrar una compra con el arreglo de ítems vacío, la respuesta es 400 `VALIDATION_ERROR` y no se crea ninguna orden.
- Al enviar un ítem con cantidad cero o negativa, la respuesta es 400 `VALIDATION_ERROR`.
- Al enviar la cantidad como texto (por ejemplo `"2"`), la respuesta es 400 `VALIDATION_ERROR`.
- Al cerrar la compra para un comprador inexistente, la respuesta es 400 `VALIDATION_ERROR` y no queda ni la cabecera ni ítem alguno.

RF-G7-03 — Consultar una orden con su detalle

- Dada una orden recién creada, al consultarla la respuesta es 200 y la cantidad de elementos de `data.items` coincide con la cantidad de ítems enviados en el cierre de compra.
- La suma de los subtotales de `data.items` es igual a `data.monto`.
- Al consultar una orden inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Orden no encontrada.".
- El orden de los ítems devueltos es estable entre consultas sucesivas.

RF-G7-04 — Recalcular una orden

- Dada una orden existente, al recalcularla con ítems distintos la respuesta es 200 y `data.monto` coincide con la suma de los nuevos ítems.
- Al consultar luego la orden con RF-G7-03, `data.items` sigue mostrando los ítems originales del checkout, no los enviados en el `PUT`, aunque `data.monto` ya sea el nuevo — una inconsistencia real a documentar, no a "corregir" en el escenario de prueba.
- Al recalcular con el arreglo de ítems vacío, la respuesta es 400 `VALIDATION_ERROR`.
- Al recalcular una orden inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G7-05 — Dar de baja una orden

- Dada una orden vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /ordenes/{id}` sobre esa orden responde 404, y deja de aparecer en RF-G7-01.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 4868/2013 de Comercio Electrónico, información previa y confirmación de la operación	Antes de confirmar, el consumidor debe conocer el detalle de lo que compra y el importe total; luego debe recibir constancia de la operación.	Cubierto: la respuesta del cierre de compra devuelve el detalle completo con subtotales y total, y el detalle de la orden queda consultable.
Ley 1334/98 de Defensa del Consumidor (SEDECO), precio cierto y veraz	El precio cobrado debe corresponder al precio informado del producto ofertado.	No implementado: no hay catálogo ni precio de referencia; el precio unitario lo fija la propia request.
Ley 1334/98, garantía de disponibilidad del producto	No debe venderse un producto sin existencias disponibles.	No implementado: no hay control de existencias ni verificación de que el producto exista.
Ley 4868/2013, derecho de retracto y cancelación	El consumidor debe poder cancelar la compra dentro del plazo previsto.	No implementado: el estado `cancelada` existe en el modelo pero no hay endpoint que lo aplique.
Marco del BCP sobre pagos en comercio electrónico	El cobro debe realizarse con un instrumento de pago válido y quedar conciliado con la orden.	No implementado: el cierre de compra no cobra; la orden nunca pasa a `pagada`.

Brechas conocidas del sandbox

- No hay catálogo, existencias ni precios de referencia: producto y precio son datos libres de la request.
- La orden nace `pendiente` y no existe ningún endpoint que la mueva a `pagada`, `enviada` o `cancelada`.
- El cierre de compra no cobra el importe ni se vincula con tarjetas, cuentas ni pagos.
- El campo `producto` de la cabecera duplica el primer ítem, lo que puede inducir a error cuando la compra tiene varios productos distintos.
- No existen descuentos, impuestos ni costos de envío en el cálculo del total.
- El listado de órdenes está limitado a 100 registros sin paginación.
- `PUT /ordenes/{id}` recalcula `producto` / `monto` de la cabecera pero nunca toca `items_orden`: tras un reemplazo, el detalle de ítems que devuelve `GET /ordenes/{id}` queda desincronizado del nuevo monto.
- Dar de baja una orden (`DELETE`) no borra ni marca sus `items_orden`, que quedan huérfanos e inaccesibles desde la API.